
Client / serveur

BARBIER Jean-Matthieu

2026-06-23

Table des matières

1	HTTP	2
1.1	Historique	2
1.1.1	HTTP/0.9	2
1.1.2	HTTP/1.0 et HTTP/1.1 (1996-1997)	2
1.1.3	HTTP/2 (2015)	3
1.2	Fonctionnement de HTTP	3
1.2.1	Rappel sur TCP/IP	3
1.2.2	Format d'un message HTTP (Requête ou Réponse)	3
2	DNS	4
2.1	Noms de domaine et hiérarchie	4
2.2	La résolution d'un nom	4
2.2.1	Le cache DNS	5
2.2.2	Quelques types d'enregistrements	5
2.3	Autorités et registrars	5
2.4	Whois	6
3	Requêtes	7
3.1	Chargement d'une page et inspection des requêtes	7
3.2	Observation d'une requête	8
3.2.1	Première ligne	9
3.2.2	URL	9
3.2.3	Méthode	10
3.2.4	En-têtes	10
4	Réponse du serveur	11
4.1	Observation d'une réponse	11
4.1.1	Code de statut	12
4.1.2	Autres en-têtes	12
4.2	Modes de fonctionnement	12
4.2.1	Serveur de fichiers	13
4.2.2	Contenu dynamique	13
5	Architecture	13
5.1	Rendu côté serveur (SSR)	13
5.2	Rendu côté client (CSR) - Single Page Application (SPA) + API	14
5.3	Hydratation (SSR + Client-Side Rendering)	15
5.4	Choix	15
6	REST	15
6.1	Concept : Ressources et Représentations	16
6.2	Méthodes HTTP = Verbes d'Action	16
6.3	Actions non REST	16
6.4	Principes Clés de REST	17

6.5	Avantages des APIs REST	17
-----	-----------------------------------	----

1 HTTP

Le protocole **HTTP** (Hypertext Transfer Protocol) est l'épine dorsale de la communication sur le World Wide Web. Il définit les règles qui permettent aux navigateurs web (clients) d'interagir avec les serveurs web pour récupérer et transmettre des informations, rendant possible la navigation sur les sites web, le téléchargement de fichiers et l'utilisation d'applications web.

1.1 Historique

1.1.1 HTTP/0.9

Comme vu précédemment, l'histoire de HTTP commence avec Tim Berners-Lee au CERN, en 1990, lors de la création des 3 piliers originels du **web** (HTML, URL, HTTP).

HTTP, plus précisément **HTTP/0.9**, est implémenté en 1991. Il était extrêmement simple, ne supportant qu'une seule méthode de requête, GET, et ne transmettait que le contenu HTML des pages. Il n'y avait pas d'en-têtes de requête ou de réponse, et la connexion TCP était fermée après chaque requête.

1.1.2 HTTP/1.0 et HTTP/1.1 (1996-1997)

L'adoption croissante du web a nécessité des améliorations significatives du protocole HTTP. En 1996, **HTTP/1.0** a introduit des changements clés :

- **nouvelles méthodes de requête** : POST et HEAD ont été ajoutées, permettant non seulement de récupérer des ressources (GET), mais aussi d'envoyer des données au serveur (POST) et de demander uniquement les en-têtes d'une ressource sans son contenu (HEAD).
- **en-têtes de requête et de réponse** : des en-têtes ont été introduits pour fournir des informations supplémentaires sur la requête et la réponse, comme le type de contenu, le navigateur utilisé, etc.

Cependant, HTTP/1.0 restait inefficace car chaque requête nécessitait l'établissement d'une nouvelle connexion TCP. En 1997, **HTTP/1.1** a apporté des améliorations majeures pour pallier à ce problème :

- **connexions persistantes (Keep-Alive)** : permet de réutiliser la même connexion TCP pour plusieurs requêtes, réduisant la latence et améliorant les performances.
- **pipelining des requêtes** : permet d'envoyer plusieurs requêtes sur la même connexion sans attendre la réponse à la première.
- **Chunked Transfer Encoding** : permet d'envoyer des données de réponse en morceaux, sans avoir à connaître la taille totale à l'avance.
- **mise en cache** : des mécanismes de cache plus sophistiqués ont été introduits pour réduire la charge sur les serveurs et améliorer la vitesse de chargement des pages.

HTTP/1.1 est resté le protocole dominant pendant plus de 15 ans, mais ses limitations en termes de performances sont devenues de plus en plus évidentes avec l'explosion du contenu web et l'augmentation du nombre d'utilisateurs.

1.1.3 HTTP/2 (2015)

HTTP/2 a été publié en 2015, basé sur le protocole Spdy de Google, et représentait une refonte majeure du protocole. Les principaux objectifs de HTTP/2 étaient d'améliorer les performances et de réduire la latence. Les innovations clés incluaient :

- **multiplexage** : possibilité d'envoyer plusieurs requêtes et réponses simultanément sur une seule connexion TCP, éliminant le problème du "head-of-line blocking" inhérent à HTTP/1.1.
- **compression des en-têtes (HPACK)** : réduit la taille des en-têtes, minimisant le volume de données transférées.
- **priorisation des requêtes** : permet au client de spécifier l'ordre dans lequel il souhaite recevoir les ressources.
- **server push** : permet au serveur d'envoyer des ressources au client avant même que celui-ci ne les demande, anticipant les besoins du client.

HTTP/2 est un protocole binaire, plus complexe à analyser manuellement qu'HTTP/1.1, mais beaucoup plus efficace en termes de performances. Il a rapidement été adopté par les principaux navigateurs web et les fournisseurs de contenu.

Les futures versions, comme HTTP/3, continuent d'explorer de nouvelles approches pour optimiser la communication web.

1.2 Fonctionnement de HTTP

1.2.1 Rappel sur TCP/IP

Avant de plonger dans HTTP, il est important de comprendre sa base : TCP/IP. TCP/IP est une suite de protocoles qui régit la communication sur Internet. HTTP, lui, s'appuie sur TCP pour établir une connexion fiable entre le client et le serveur.

- **IP (Internet Protocol)** : responsable de l'adressage et du routage des paquets de données.
- **TCP (Transmission Control Protocol)** : assure une transmission fiable des données, avec vérification d'erreurs et réassemblage des paquets dans l'ordre correct. HTTP utilise TCP car il a besoin de garantir que les données arrivent intégralement et dans l'ordre.

En bref, HTTP "parle" à travers TCP/IP. TCP/IP gère le *transport* des données, HTTP gère le *sens* de ces données.

1.2.2 Format d'un message HTTP (Requête ou Réponse)

Un message HTTP (que ce soit une requête ou une réponse) est un bloc de texte brut structuré. Il comprend les éléments suivants, séparés par des lignes vides :

1. ligne de début :

- Pour une requête : `MÉTHODE URI Version` (ex: `GET /index.html HTTP/1.1`)
- Pour une réponse : `Version CodeStatut Phrase` (ex: `HTTP/1.1 200 OK`)

2. en-têtes (Headers) : une série de paires clé-valeur, chacune sur une ligne distincte, séparées par un deux-points (:). Les en-têtes fournissent des métadonnées sur le message. Exemple :

```
Content-Type: text/html
Content-Length: 1234
User-Agent: Mozilla/5.0
```

3. **ligne vide** : sépare les en-têtes du corps du message.
4. **Corps (Body)** : optionnel, contient les données réelles de la requête (ex : données de formulaire POST) ou de la réponse (ex : code HTML).

2 DNS

Sur le réseau, les machines ne se connaissent pas par leur nom mais par leur **adresse IP** (par exemple 2001:470:142:5::116 ou 208.118.235.148). Or, dans une URL, on écrit un **nom de domaine** lisible par un humain : `www.gnu.org`. Il faut donc **traduire ce nom en adresse IP** avant toute connexion : c'est le rôle du **DNS** (Domain Name System), une sorte d'**annuaire géant** d'Internet.



Une analogie

Le DNS fonctionne comme le répertoire d'un téléphone : on connaît le **nom** de la personne (`www.gnu.org`), et le répertoire nous donne son **numéro** (l'adresse IP) pour pouvoir l'appeler.

2.1 Noms de domaine et hiérarchie

Un nom de domaine est **hiérarchique**, et se lit **de droite à gauche**. Chaque partie, séparée par un point, correspond à un niveau de la hiérarchie :

```
www . gnu . org .
|      |      |      |
|      |      |      +-- racine (le point final, souvent omis)
|      |      +----- TLD : domaine de premier niveau (.org)
|      +----- domaine (gnu)
+----- sous-domaine (www)
```

- la **racine** (*root*), notée par un point final (presque toujours omis), est le sommet de l'arbre;
- les **TLD** (*Top-Level Domains*, domaines de premier niveau) : les extensions comme `.org`, `.fr`, `.com`...;
- le **domaine** proprement dit (`gnu`), enregistré par une organisation;
- les **sous-domaines** (`www`, `mail`, `philosophy`...), librement définis par le propriétaire du domaine.

2.2 La résolution d'un nom

La **résolution** est l'opération qui transforme un nom de domaine en adresse IP. Elle a lieu **avant** la requête HTTP. De façon simplifiée :

1. le navigateur demande à un **résolveur DNS** (un serveur, souvent celui de la box ou du fournisseur d'accès) : « quelle est l'adresse IP de `www.gnu.org`? »;
2. si le résolveur ne connaît pas la réponse, il interroge à son tour la **hiérarchie** des serveurs DNS :
 - un **serveur racine** lui indique quel serveur gère le TLD `.org`;
 - le **serveur du TLD .org** lui indique quel serveur **fait autorité** pour `gnu.org`;
 - le **serveur faisant autorité** pour `gnu.org` répond enfin l'adresse IP de `www.gnu.org`;
3. l'adresse IP **remonte** jusqu'au navigateur, qui peut alors ouvrir la connexion.



En pratique

La résolution DNS utilise généralement le protocole **UDP** sur le **port 53**. On peut l'observer en ligne de commande :

```
nslookup www.gnu.org
dig www.gnu.org
host www.gnu.org
```

2.2.1 Le cache DNS

Refaire tout ce trajet à chaque clic serait long. Le résultat d'une résolution est donc **mis en cache** (mémorisé un certain temps) à plusieurs niveaux : dans le **navigateur**, dans le **système d'exploitation**, et dans les **résolveurs** intermédiaires. C'est en partie pourquoi une page déjà visitée se charge plus vite.

La durée de conservation est fixée par le **TTL** (*Time To Live*) associé à chaque enregistrement : c'est le propriétaire du domaine qui le définit. Un TTL court permet de propager rapidement un changement (par exemple un changement de serveur), au prix de résolutions plus fréquentes.

2.2.2 Quelques types d'enregistrements

Un serveur DNS ne stocke pas qu'une correspondance nom → IP. Chaque domaine possède des **enregistrements** (*records*) de différents types :

- A : adresse **IPv4** d'un nom
- AAAA : adresse **IPv6** d'un nom
- CNAME : **alias** d'un nom vers un autre nom
- MX : serveur de **courrier électronique** du domaine
- NS : serveur(s) **faisant autorité** pour le domaine
- TXT : texte libre (souvent utilisé pour la vérification et la sécurité des mails)

2.3 Autorités et registrars

Qui décide que `gnu.org` pointe vers telle adresse IP? L'organisation du DNS repose sur une **chaîne d'autorités** :

- l'**ICANN** (et l'IANA) coordonne le système au sommet : la racine et l'attribution des TLD;
- chaque **TLD** est géré par un **registre** (*registry*). Par exemple, l'`.fr` est géré par l'**Afnic**, le `.org` par **PIR** (*Public Interest Registry*);

- on n'achète pas un domaine directement auprès du registre, mais par l'intermédiaire d'un **bureau d'enregistrement** (*registrar*) : OVH, Gandi, Namecheap... C'est lui qui, pour un abonnement annuel, **enregistre** le domaine à votre nom auprès du registre et vous laisse configurer ses enregistrements DNS;
- le **titulaire** (*registrant*) est la personne ou l'organisation à qui le domaine est attribué.

**On ne « possède » pas un domaine**

Un nom de domaine n'est pas acheté définitivement : il est **loué** (en général à l'année). Si l'abonnement n'est pas renouvelé, le domaine **expire** et redevient disponible pour quelqu'un d'autre.

2.4 Whois

Whois (« qui est-ce ? ») est un service public permettant de **consulter les informations d'enregistrement** d'un nom de domaine : son registrar, ses dates de création et d'expiration, ses serveurs de noms (NS), et — selon les cas et la réglementation — des informations sur le titulaire.

On l'interroge avec la commande `whois`, ou via des sites web dédiés :

```
whois gnu.org
```

donne (extrait) des informations du type :

```
Domain Name: GNU.ORG
Registry Domain ID: ...
Registrar: Gandi SAS
Creation Date: 1990-...
Registry Expiry Date: 20XX-...
Name Server: NS1.GNU.ORG
...
```

**Vie privée**

Historiquement, le Whois exposait nom, adresse et e-mail du titulaire. Depuis le **RGPD** et la généralisation des services d'**anonymisation** (le registrar masque les coordonnées personnelles), ces informations sont aujourd'hui souvent remplacées par celles d'un service de protection de la vie privée.

admonish note title="Programme" - SNT : thème Web

3 Requêtes

3.1 Chargement d’une page et inspection des requêtes

En prenant un exemple sur un chargement de page <https://www.gnu.org/philosophy/philosophy.html>, nous allons voir comment fonctionne une requête HTTP en utilisant les outils d’inspection du navigateur : F12 > Network.

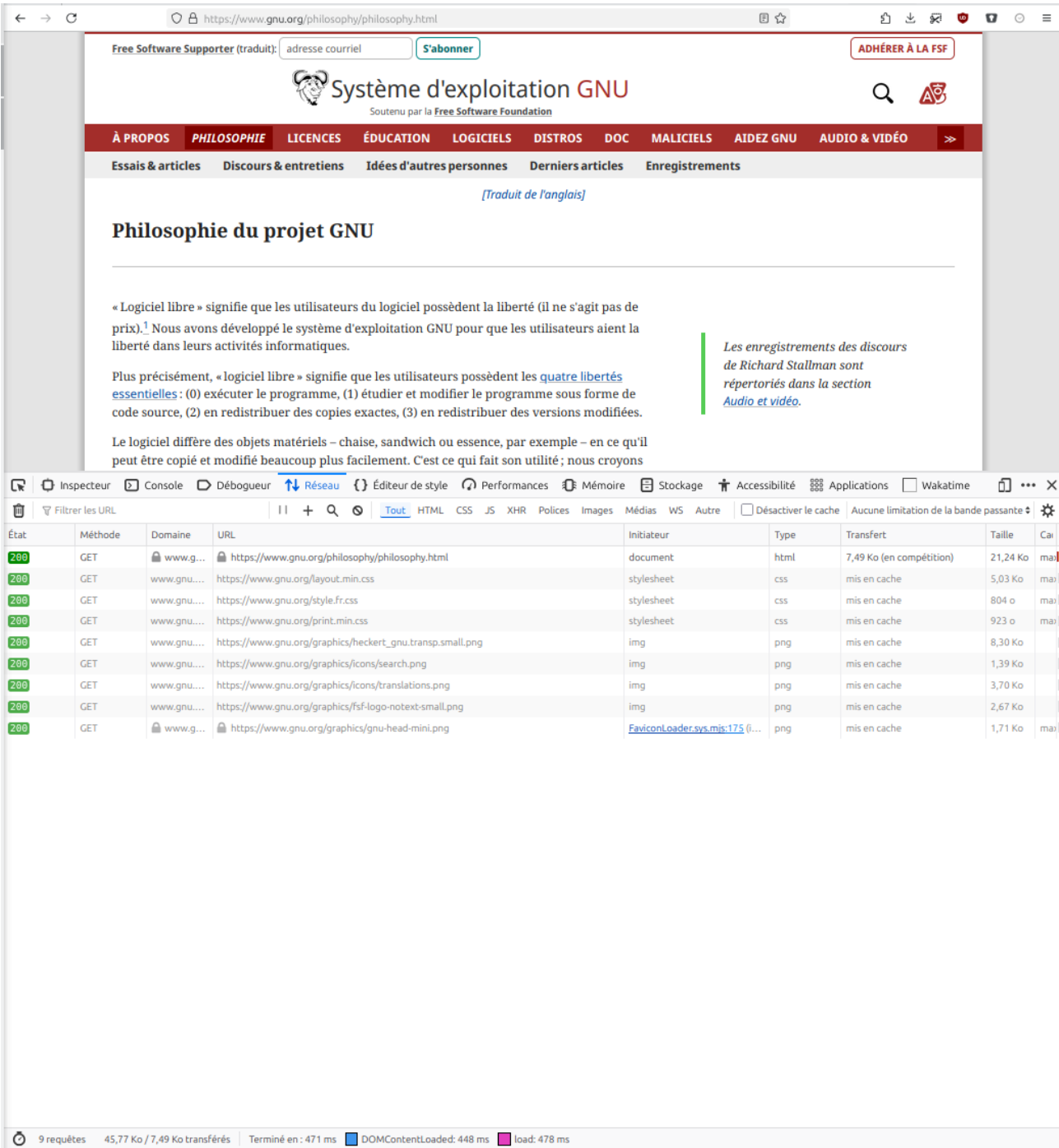


FIGURE 1 – Trafic réseau gnu.org

On peut observer que le chargement de cette page nécessite plusieurs requêtes successives ou simultanées.

3.2 Observation d'une requête

Nous allons nous intéresser plus particulièrement à la première requête.

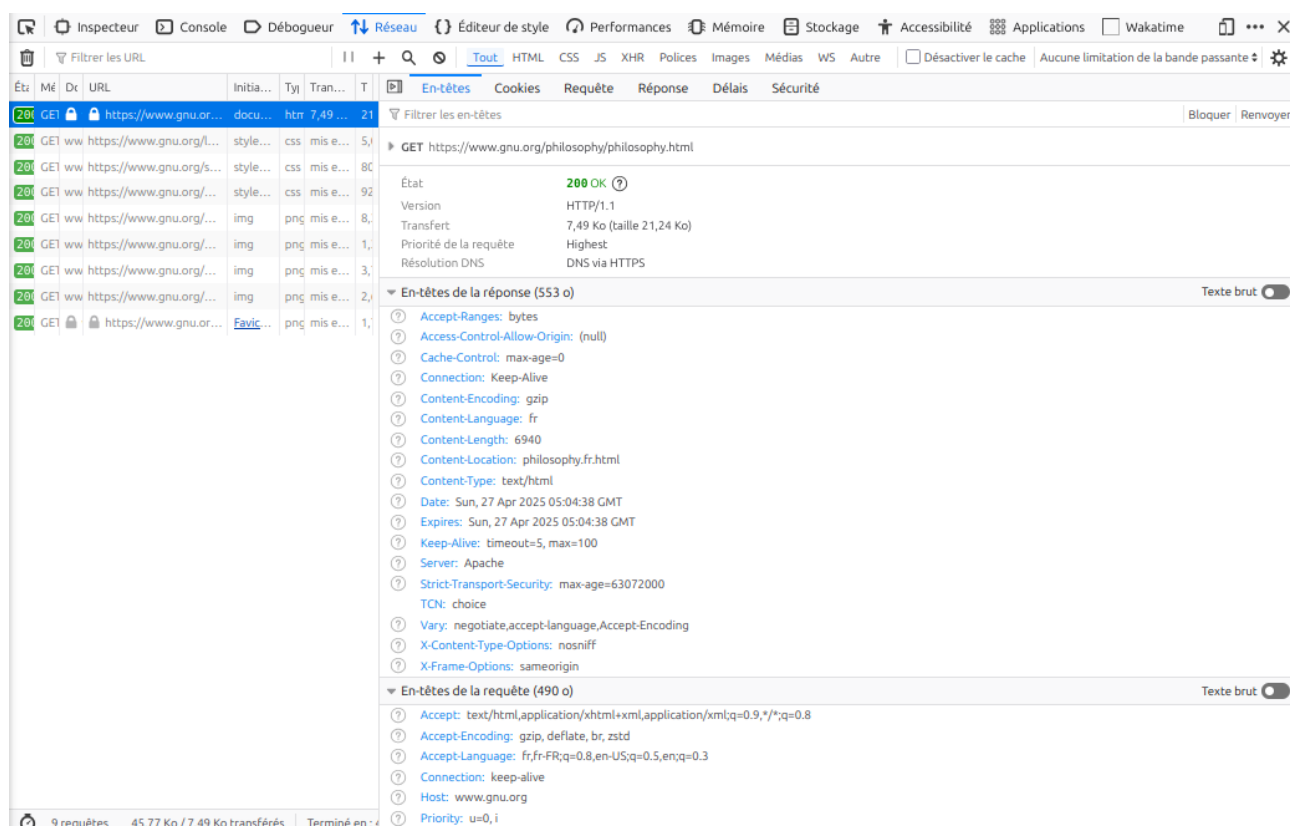


FIGURE 2 – Requête philosophie.html

L'affichage indique à la fois la requête et la réponse.

Dans le protocole HTTP, la requête est faite sous forme d'un texte "spécial". Une première partie (jusqu'à la première ligne vide) constitue les en-têtes, et la deuxième partie constitue les données.

Cette première partie d'en-têtes est recopiées ci-dessous :

```
GET /philosophy/philosophy.html HTTP/1.1
```

```
Host: www.gnu.org
```

```
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
```

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
```

```
Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3
```

```
Accept-Encoding: gzip, deflate, br, zstd
```

```
Connection: keep-alive
```

```
Upgrade-Insecure-Requests: 1
```

```
Sec-Fetch-Dest: document
```

```
Sec-Fetch-Mode: navigate
```

```
Sec-Fetch-Site: none
```

```
Sec-Fetch-User: ?1
```

```
Priority: u=0, i
```

3.2.1 Première ligne

C'est une requête du **client**, qui part donc de notre navigateur, en direction de son destinataire. Mais à qui est envoyée cette requête ?

3.2.2 URL

Petit rappel sur les URL (vu précédemment) :

L'URL `https://www.gnu.org/philosophy/philosophy.html` peut être décomposée en plusieurs parties :

- `https://` indique le protocole (protocol), sécurisé ou non (le `s`), utilisé pour se connecter au site
- `www.gnu.org` est le nom de domaine (hostname), que le **DNS** (Domain Name System) traduit en une adresse IP
- `/philosophy/philosophy.html` est le chemin (path / pathname)

Une url peut aussi contenir d'autres éléments : le port, une partie "search" ou "query string", un "hash"...

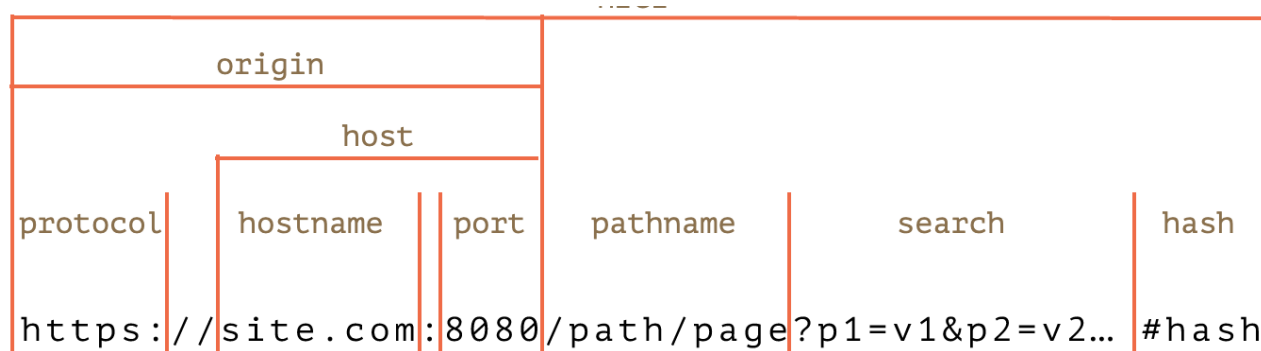


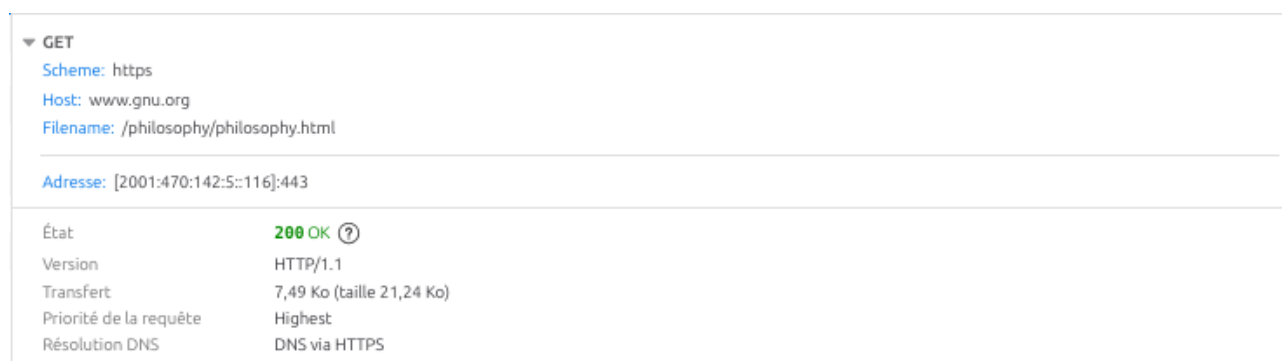
FIGURE 3 – URL

Le nom de domaine `www.gnu.org` n'est pas une adresse réseau : il doit d'abord être **traduit en adresse IP** par le **DNS**, avant même que la requête HTTP ne parte. Cette résolution de nom, ainsi que l'organisation du DNS (hiérarchie, autorités, registrars, whois), est détaillée dans la partie DNS.

La requête est donc envoyée vers l'adresse IP correspondant au *hostname*, sur le port précisé dans l'url, et si aucun port n'est précisé (ce qui est le cas la plupart du temps), vers le port 80 pour le http et 443 pour le https.

Ici, la requête est donc envoyée vers l'IPv6 `[2001:470:142:5::116]` sur le port 443 (on peut le voir dans la partie General du détail) :

Adresse: `[2001:470:142:5::116]:443`

**FIGURE 4** – Infos générales

La première ligne de la requête :

GET /philosophy/philosophy.html HTTP/1.1

indique

- la **méthode** : GET
- le chemin (pathname) : /philosophy/philosophy.html
- le protocole utilisé et sa version (ici HTTP version 1.1)

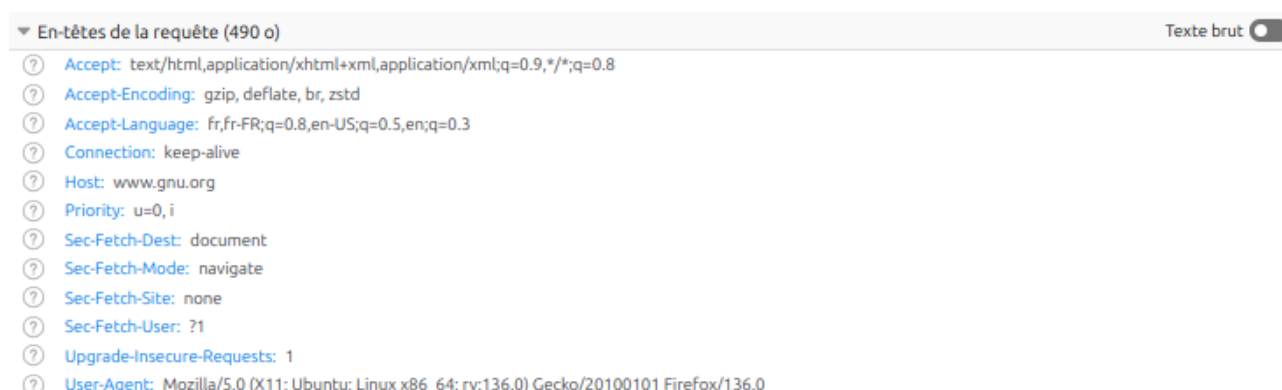
3.2.3 Méthode

La méthode est en fait une commande spécifiant un type de requête, c'est-à-dire qu'elle demande au serveur d'effectuer une action sur la ressource indiquée juste ensuite. Les différentes méthodes disponibles sont GET, POST, HEAD, OPTIONS, DELETE, PATCH, PUT et plus rarement CONNECT et TRACE.

Une requête avec la méthode **GET** est une demande pour récupérer une ressource; de manière générale c'est une méthode qui doit correspondre à une opération *idempotente* (qui peut être répétée plusieurs fois sans que cela aie un effet)

3.2.4 En-têtes

D'autres informations sont transmises au serveur dans les en-têtes; toutes ces informations sont optionnelles, la seule obligatoire est cette première ligne.

**FIGURE 5** – Infos générales

On peut reconnaître :

- **Host** : quel était l'hôte (nom de domaine et port) à qui la requête a été envoyé
- **Accept** : quel(s) types de document le navigateur accepte de recevoir en réponse à cette requête, éventuellement avec un ordre de priorité (le q=)
- **Accept-Encoding** : le navigateur accepte (ou non) les données compressées
- **Accept-Language** : quelles sont la ou les langues préférées par l'utilisateur de ce navigateur (avec un ordre de priorité donné par le q=)
- **Cache-Control** : indications pour garder ou non la réponse en "cache" (mémoire locale ou sur le réseau)
- **User-Agent** : une chaîne de caractères indiquant le système d'exploitation de la machine faisant tourner le navigateur, des infos sur le navigateur et sa version.
- ...

4 Réponse du serveur

Le serveur est une machine qui fait tourner un programme (service) qui écoute sur une adresse IP et un port, et qui répond aux demandes des clients.

La réponse commence par un ou plusieurs en-têtes, puis une ligne blanche et enfin les données de la réponse.

4.1 Observaton d'une réponse

On peut observer cette réponse dans l'inspecteur :

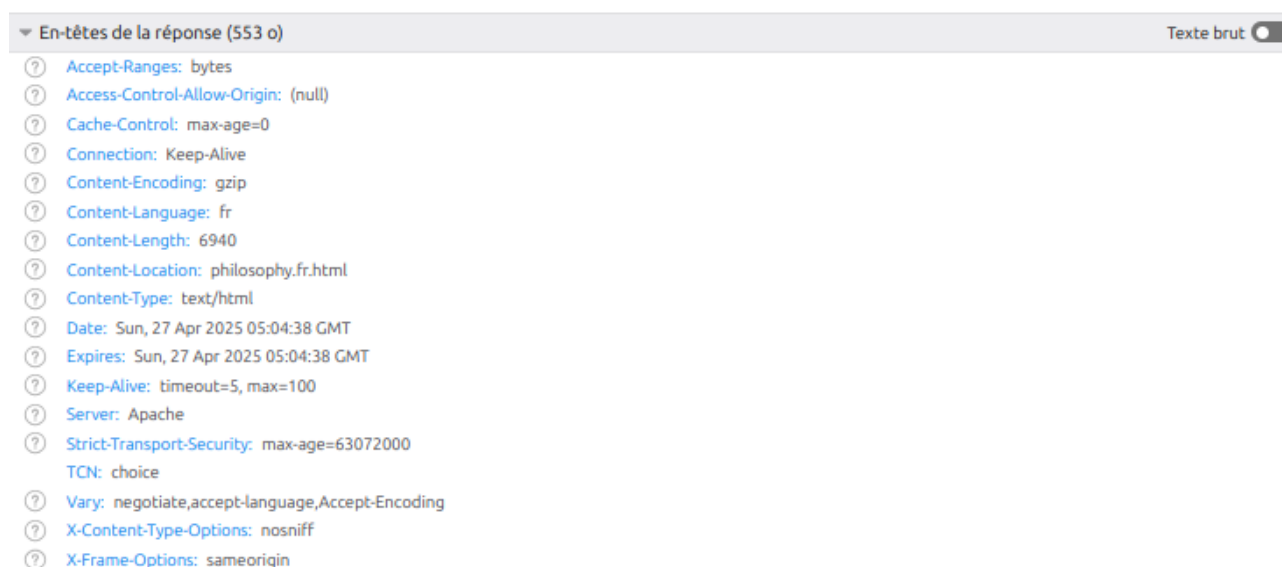


FIGURE 6 – Réponse - parsée

ou sous sa forme brute :

HTTP/1.1 200 OK

Date: Sun, 27 Apr 2025 05:04:38 GMT
Server: Apache
Content-Location: philosophy.fr.html
Vary: negotiate,accept-language,Accept-Encoding
TCN: choice
Strict-Transport-Security: max-age=63072000
X-Frame-Options: sameorigin
X-Content-Type-Options: nosniff
Access-Control-Allow-Origin: (null)
Accept-Ranges: bytes
Cache-Control: max-age=0
Expires: Sun, 27 Apr 2025 05:04:38 GMT
Content-Encoding: gzip
Content-Length: 6940
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html
Content-Language: fr

4.1.1 Code de statut

La première ligne indique la version du protocole utilisé (HTTP version 1.1 ici), un code (ici 200) et un état (OK)

- Les codes **1xx** indiquent une information
- Les codes **2xx** indiquent un succès
- Les codes **3xx** indiquent une redirection
- Les codes **4xx** et **5xx** indiquent une erreur

Liste détaillée sur https://fr.wikipedia.org/wiki/Liste_des_codes_HTTP

4.1.2 Autres en-têtes

Le reste des en-tête donne des informations sur le contenu renvoyé et la manière dont il doit être traité :

- **Content-Type** : type de contenu
- **Date** : date et heure de la réponse
- **Content-Length** : taille du contenu en octets
- **Content-Language** : information sur la langue du contenu
- ...

4.2 Modes de fonctionnement

On peut distinguer deux “modes” de fonctionnement :

- soit le serveur renvoie le contenu de fichiers présents “physiquement” sur la machine
- soit le serveur renvoie un contenu qu’il génère au vol, dynamiquement

4.2.1 Serveur de fichiers

Un serveur de fichiers va associer un *chemin* à un *répertoire* :

Exemple : sur un serveur écoutant sur le port http par défaut (80) d'une machine dont l'ip est correspond au domaine `example.com`, on associe le chemin `/static/` est associé au répertoire `/var/www/html/`

Dans ce cas

- la requête `http://example.com/static/image.jpg` cherche et renvoie l'image `/var/www/html/image.jpg`
- la requête `http://example.com/static/test/toto.txt` cherche et renvoie le fichier texte `/var/www/html/test/toto.txt`

si un fichier n'existe pas avec le chemin correspondant, le serveur renvoie un code d'erreur 404 (Not found - Non trouvé)

4.2.2 Contenu dynamique

Le serveur va associer un morceau de programme (une fonction, par exemple) à un chemin ou groupe de chemins donnés, et renverra le résultat de ce programme.

Exemple : sur un serveur écoutant sur le port http par défaut (80) d'une machine dont l'ip est correspond au domaine `example.com`, on associe le chemin `/bonjour` à une fonction `bonjour()` qui retourne la chaîne de caractères "hello world"

- la requête `http://example.com/bonjour` renverra donc "hello world"
- la requête `http://example.com/boujour` renverra une erreur 404

5 Architecture

Une application web peut être développée en utilisant plusieurs architectures :

- rendu côté serveur (Server-Side Rendering - SSR)
- rendu côté client (Client-Side Rendering - CSR) - application à page unique (Single Page Application - SPA)
- application hybride : hydratation (SSR + CSR)

Le choix de l'architecture dépend de plusieurs facteurs.

5.1 Rendu côté serveur (SSR)

Modèle "traditionnel" : le serveur génère l'intégralité du HTML pour chaque requête utilisateur. Le navigateur reçoit une page HTML complète et l'affiche directement. Chaque interaction avec le serveur renvoie une nouvelle page HTML.

Avantages :

- **SEO Amélioré** : Les moteurs de recherche peuvent facilement indexer le contenu HTML.
- **Temps d'affichage initial rapide** (First Contentful Paint) : Le navigateur reçoit directement du contenu affichable.

- **Compatibilité** : Fonctionne sur tous les navigateurs, même anciens, sans nécessiter JavaScript.

Inconvénients :

- **Charge serveur plus élevée** : Le serveur doit générer le HTML pour chaque requête, ce qui peut être coûteux en ressources.
- **Moins interactif** : Chaque interaction nécessite une nouvelle requête au serveur, ce qui peut entraîner des rechargements de page.
- **Moins flexible** : La logique de présentation est étroitement liée au serveur, données et présentation sont mélangées.

Technologies courantes :

- PHP (Laravel, Symfony)
- Python (Django, Flask, Bottle)
- Ruby (Ruby on Rails)
- Java (Spring)

5.2 Rendu côté client (CSR) - Single Page Application (SPA) + API

Modèle plus “moderne” : le navigateur fait le travail de génération de la page, en fonction des données récupérées sur le serveur via une **API** (application programming interface). Présentation et données sont séparées.

Fonctionnement : le navigateur télécharge une page HTML unique qui contient le squelette de l’application et le code JavaScript nécessaire. Les interactions de l’utilisateur sont gérées par JavaScript, qui met à jour dynamiquement le contenu de la page en utilisant des appels API au serveur pour récupérer ou enregistrer des données. Avantages :

- **expérience utilisateur fluide** : pas de rechargement de page pour les interactions, ce qui offre une expérience plus réactive et agréable, et la création d’interfaces plus dynamiques.
- **rapidité** : une fois la page initiale chargée, les requêtes API sont généralement plus rapides que les rechargements de page complets.
- **séparation des logiques** : la logique de présentation (JavaScript) est séparée de la logique métier (API), ce qui facilite le développement et la maintenance.
- **facilité de maintenance** : l’API et l’interface utilisateur peuvent être développées et maintenues indépendamment.

Inconvénients :

- **SEO (Search Engine Optimization) potentiellement plus difficile** : les moteurs de recherche peuvent avoir du mal à indexer le contenu généré dynamiquement par JavaScript (bien que certains moteurs de recherche puisse indexer les pages dynamiques).
- **temps d’affichage initial plus long** : le téléchargement initial de la page unique et de tout le code JavaScript peut prendre du temps, impactant le temps d’affichage initial, l’expérience utilisateur et le SEO.
- **dépendance JavaScript** : nécessite JavaScript activé dans le navigateur.

Technologies Courantes :

- Frameworks JavaScript/Typescript (React, Angular, Vue.js) pour le frontend

- Tout langage serveur pour l'API (Python, Node.js, Java, PHP, etc.)

5.3 Hydratation (SSR + Client-Side Rendering)

Fonctionnement : combine les avantages du rendu côté serveur (SSR) et du rendu côté client (SPA). Le serveur génère initialement le HTML de la page, comme dans le SSR. Ensuite, le code JavaScript est téléchargé et “hydrate” le HTML, prenant le contrôle de l'interaction avec l'utilisateur et gérant les mises à jour dynamiques comme dans une SPA.

Avantages :

- **meilleur SEO (Search Engine Optimization)** : le contenu est initialement indexable par les moteurs de recherche.
- **temps d'affichage initial rapide** : le navigateur reçoit du contenu affichable rapidement grâce au SSR.
- **expérience utilisateur fluide** : les interactions sont gérées de manière réactive par le JavaScript côté client.

Inconvénients :

- **complexité** : plus complexe à mettre en œuvre que le SSR ou la SPA.
- **charge serveur** : nécessite une capacité serveur pour le rendu initial.
- **temps d'hydratation** : un certain temps est nécessaire pour que JavaScript prenne le contrôle de l'application, ce qui peut être perceptible pour l'utilisateur.

Technologies Courantes :

- La plupart des frameworks JavaScript modernes prennent en charge l'hydratation, y compris React, Vue.js et Angular.

5.4 Choix

Il n'y a pas de “meilleure” approche universelle. Le choix dépend du contexte. Pour les applications axées sur le contenu et nécessitant un bon référencement, le SSR ou l'hydratation sont de bons choix. Pour les applications très interactives où l'expérience utilisateur est primordiale, une SPA + API peut être plus appropriée. L'hydratation offre un bon compromis entre les deux, mais au prix d'une complexité accrue.

6 REST

Références :

- https://fr.wikipedia.org/wiki/Representational_state_transfer

Les API **REST** (Representational State Transfer) sont capitales dans le développement web moderne, en facilitant l'interconnexion entre différentes applications et services. Elles permettent à des applications d'échanger des données de manière standardisées, même si elles sont écrites dans des langages différents ou fonctionnent sur des plateformes distinctes.

6.1 Concept : Ressources et Représentations

Au cœur d'une API REST réside le concept de "ressource". Une ressource peut être n'importe quoi identifiable : un utilisateur, un produit, un article de blog, une commande, etc. Chaque ressource est identifiée par une **URL unique**, agissant comme son adresse sur le réseau. Exemple : `https://api.example.com/utilisateur/123` où 123 est l'identifiant de l'utilisateur.

Une API REST ne transmet pas directement les données elles-mêmes. Elle transmet une *représentation* de ces données. Cette représentation peut prendre différentes formes, les plus courantes étant **JSON** (JavaScript Object Notation) et **XML**. Le choix du format de représentation est généralement spécifié dans les en-têtes de la requête et de la réponse.

6.2 Méthodes HTTP = Verbes d'Action

Les interactions avec une API REST s'effectuent à l'aide des méthodes HTTP standard. Ces méthodes définissent l'action à effectuer sur la ressource :

- **GET** : Récupérer une ressource. Par exemple, `GET /utilisateurs/123` pour obtenir les informations de l'utilisateur avec l'ID 123.
- **POST** : Créer une nouvelle ressource. Par exemple, `POST /utilisateurs` pour créer un nouvel utilisateur. Les données du nouvel utilisateur sont incluses dans le corps de la requête.
- **PUT** : Mettre à jour une ressource existante en remplaçant complètement son contenu. Par exemple, `PUT /utilisateurs/123` pour mettre à jour l'ensemble des informations de l'utilisateur 123.
- **PATCH** : Mettre à jour une ressource existante en modifiant partiellement son contenu. Par exemple, `PATCH /utilisateurs/123` pour modifier uniquement le nom de l'utilisateur 123.
- **DELETE** : Supprimer une ressource. Par exemple, `DELETE /utilisateurs/123` pour supprimer l'utilisateur 123.

6.3 Actions non REST

Bien que la méthode **POST** soit principalement associée à la création de nouvelles ressources, elle peut également être utilisée pour déclencher des actions spécifiques sur une ressource ou effectuer des opérations qui ne s'inscrivent pas dans les verbes standard. Par exemple, `POST /utilisateurs/123/messages` pourrait être utilisé pour envoyer un message à l'utilisateur 123, sans pour autant créer une nouvelle ressource "message" indépendante.

Cela permet de réaliser des opérations personnalisées ou complexes, ou de lancer des processus asynchrones liés à une ressource existante, en exploitant le corps de la requête pour transmettre les paramètres nécessaires à l'action. Il est important de documenter clairement ces utilisations non conventionnelles de POST pour assurer la clarté de l'API.

Dans la pratique, cette approche permet de gérer des opérations à la manière d'une API RPC (Remote Procedure Call), où l'on peut appeler des fonctions distantes en utilisant des requêtes HTTP. Même si "c'est pas bien", cela peut être indispensable dans beaucoup de situations.

6.4 Principes Clés de REST

L'architecture REST s'appuie sur plusieurs principes pour garantir son efficacité et sa scalabilité :

- **Stateless (Sans État)** : chaque requête du client au serveur doit contenir toutes les informations nécessaires pour être traitée. Le serveur ne conserve aucune information sur les requêtes précédentes.
- **Cacheable (Mise en Cache)** : les réponses du serveur doivent être explicitement définies comme pouvant être mises en cache ou non, ce qui permet d'améliorer les performances.
- **Uniform Interface (Interface Uniforme)** : l'utilisation des méthodes HTTP standard (GET, POST, PUT, DELETE) et d'une structure d'URL cohérente permet de simplifier l'interaction avec l'API.

6.5 Avantages des APIs REST

- **simplicité** : facile à comprendre et à utiliser grâce à l'utilisation des méthodes HTTP standard.
- **documentation** : les API REST peuvent être facilement documentées (voire auto-documentées) à l'aide d'outils comme Swagger ou OpenAPI, facilitant la compréhension et l'utilisation par les développeurs.
- **scalabilité** : l'absence d'état du serveur permet une scalabilité horizontale facile.
- **flexibilité** : supporte différents formats de données et peut être utilisée avec différents langages de programmation.
- **interopérabilité** : permet l'intégration facile entre différentes applications et services.

Les APIs REST constituent une approche puissante et largement adoptée pour la création d'applications web modernes, favorisant l'interopérabilité et la modularité. Leur utilisation simplifie l'intégration de services tiers et permet la construction d'architectures distribuées robustes et évolutives.